

Practical:1

Aim:

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Mergesort and Quicksort

1. Bubble sort Algorithm:

Bubble Sort:-

Input :- Array A with n elements.
Output :- Sorted array A in ascending order.

Step 1:- Start.

Step 2:- Read 'n' and array 'A'

Step 3:- Set $i = 0$

Step 4:- Repeat while $i < n - 1$
Set $j = 0$
Compare $A[j]$ and $A[j+1]$
If $A[j] > A[j+1]$, swap them.
Increment j until $j < n - i - 1$
Increment i

Step 5:- Display Sorted Array

Step 6:- Stop.

Program code:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {5, 3, 8, 4, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    bubbleSort(arr, n);

    cout << "\nAfter sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

Before sorting: 5 3 8 4 2
After sorting: 2 3 4 5 8

Time complexity:

Time complexity:-

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 2 + 1$$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

$$T(n) = n^2$$

$$T(n) = O(n^2)$$

Best case:-
Only 1 pass is made through the array.

$$T(n) = n - 1$$

$$T(n) = n - 1 + (n - 1) - 1 + \dots$$

$$T(n) = O(n)$$

Average case:-

Comparisons = $n(n-1)/2$
Swaps = $n(n-1)/4$

$$T(n) = n(n-1)/2 + n(n-1)/4$$

$$T(n) = \frac{3}{4}n^2 - \frac{3}{4}n$$

$$= n^2$$

$$T(n) = O(n^2)$$

Worst case:-

$$\text{Comparison} = (n-1) + (n-2) + \dots + 1$$

$$= \frac{n(n-1)}{2}$$

$$T(n) = (2+2) + 3 \cdot \left\{ \frac{n(n-1)}{2} \right\}$$

$$= n^2 - n + 2n + 2$$

$$= n^2 + n + 2$$

$$= n^2$$

$$T(n) = O(n^2)$$

Conclusion:

Bubble sort sorts the array by repeatedly comparing and swapping adjacent elements. It arranges the elements in ascending order after completing all passes.

2. Selection sort Algorithm:

Selection Sort:-
Input Array:- Array A with n elements.
Output:- A sorted in ascending order.

Step 1:- For $i \leftarrow 0$ to $n-2$.
 $min \leftarrow i$

Step 2:- For $j \leftarrow i+1$ to $n-1$

Step 3:- If $a[j] < a[min]$ then.
 $min \leftarrow j$
 END If
 END For.

Step 4:- $Temp \leftarrow a[i]$
 $a[i] \leftarrow a[min]$
 $a[min] \leftarrow Temp$

Step 5:- END For

Step 6:- Display the Sorted Array

Step 7:- Stop.

Program code:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
    }
}
```

```
}  
// Swap  
temp = arr[i];  
arr[i] = arr[minIndex];  
arr[minIndex] = temp;  
}  
}  
  
int main() {  
    int arr[] = {64, 25, 12, 22, 11};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    cout << "Before sorting: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    selectionSort(arr, n);  
  
    cout << "\nAfter sorting: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

```
Before sorting: 64 25 12 22 11  
After sorting: 11 12 22 25 64
```

Time complexity:

Selection Sort
Time Complexity :-

Best case:- Array already sorted.

Outer loop $\rightarrow n$ times
Inner loop $\rightarrow T(n) = (n-1) + (n-2) + \dots + 2 + 1$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

Worst case:- Array sorted in reverse order.

Comparisons $= \frac{n(n-1)}{2}$
Swaps $= 0$

$$T(n) = (n-1) + (n-2) + \dots + 2 + 1$$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

Average case:-

Comparisons $\bullet T(n) = (n-1) + (n-2) + \dots + 2 + 1$
Swaps $= n - 1$

$$= \frac{n(n-1)}{2} + n$$

$$= \frac{n^2 - n}{2} + n$$

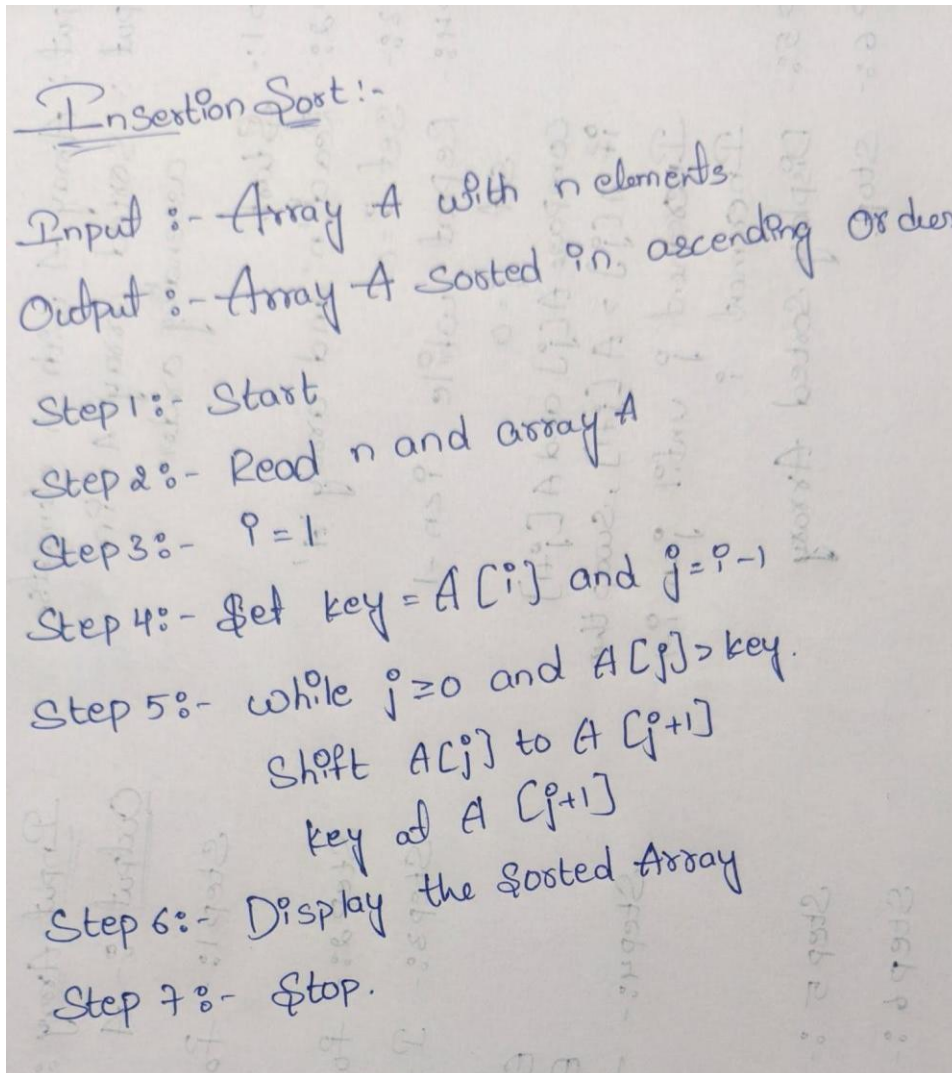
$$T(n) = n^2$$

$$[T(n) = O(n^2)]$$

Conclusion:

Selection sort sorts the array by repeatedly finding the smallest element from the unsorted portion and placing it in its correct position. It arranges the elements in ascending order after completing all passes.

3.Insertion sort Algorithm:



Program:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
```

```
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;

    int arr[100]; // fixed size for simplicity
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Before sorting: ";
    printArray(arr, n);

    insertionSort(arr, n);

    cout << "After sorting: ";
    printArray(arr, n);

    return 0;
}
```

Output:

```
Enter size of array: 6
Enter 6 elements: 9 4 7 2 5 1
Before sorting: 9 4 7 2 5 1
After sorting: 1 2 4 5 7 9
```

Time complexity:

Insertion Sort.

Best Case:- Array already sorted
Only 1 comparison per outer iteration. $n-1$
 $T(n) = n-1$
 $T(n) = n$
 $T(n) = O(n)$

Worst Case:- Array sorted in reverse order.
Comparisons = $1 + 2 + 3 + \dots + (n-1)$
 $= \frac{n(n-1)}{2}$
 $T(n) = n + \frac{n(n-1)}{2}$
 $= \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

Average Case:-
Comparisons = $\frac{n(n-1)}{4}$
 $T(n) = \frac{n + \frac{n(n-1)}{4}}{4}$
 $= \frac{n + \frac{n^2 - n}{4}}{4}$
 $T(n) = \frac{n^2}{4}$
 $T(n) = O(n^2)$

Conclusion:

Insertion sort sorts the array by inserting each element into its correct position in the sorted portion of the array. It arranges the elements in ascending order after completing all passes.

4. Merge sort Algorithm:

Merge Sort:-

Input :- Array A with n element

Output :- Array A sorted in ascending order.

Step 1:- If $low \leftarrow high$ then
 $mid \leftarrow (low + high) / 2$

Step 2:- Merge sort (A, low, mid)
Merge sort (A, mid + 1, high)
Merge sort (A, low, mid, high)

Step 3:- END If
END

Step 4:- Merge (A, low, mid, high)
Merge the two sorted subarray into a temporary array copy the temporary array back into A

Step 5:- END.

Program:

```
#include <iostream>
using namespace std;

void merge(int arr[], int low, int mid, int high) {
    int temp[100];
    int i = low, j = mid + 1, k = low;

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high) {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = low; i <= high; i++) {
        arr[i] = temp[i];
    }
}
```

```
void mergeSort(int arr[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
        mergeSort(arr, low, mid);  
        mergeSort(arr, mid + 1, high);  
        merge(arr, low, mid, high);  
    }  
}
```

```
int main() {  
    int arr[] = {38, 27, 43, 3, 9, 82, 10};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    cout << "Before sorting: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    mergeSort(arr, 0, n - 1);  
  
    cout << "\nAfter sorting: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

```
Before sorting: 38 27 43 3 9 82 10  
After sorting: 3 9 10 27 38 43 82
```

Time complexity:

Merge Sort:-

All cases (Best = Worst = Average):-

Merge sort always splits the array exactly in half and always merges in $O(n)$, regardless of input order.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T(n) = 2[2T\left(\frac{n}{4}\right) + \frac{n}{2}] + n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

$$= 8T\left(\frac{n}{8}\right) + 3n$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$k = \log_2 n$$

$$T(n) = nT(1) + n \log_2 n$$

$$= n + n \log n$$

$$\boxed{T(n) = O(n \log n)}$$

Conclusion:

Merge sort sorts the array by dividing it into smaller subarrays and merging them after sorting. It efficiently arranges the elements in ascending order with a time complexity of $O(n \log n)$.

5. Quicksort Algorithm:

Input :- Array Element.
Output :- Sorted Array.

Step 1:- Quick sort (A, low, high)

Step 2:- If low < high then.
Pivot $\leftarrow$ Position (A, low, high)

Step 3:- Quick sort (A, low, Pivot - 1)
Quick sort (A, Pivot + 1, high)

Step 4:- END If

Step 5:- Partition (A, low, high)

Step 6:- Pivot $\leftarrow$ A [low]
i $\leftarrow$ low
j $\leftarrow$ high

Step 7:- while i < j
while A[i] $\leq$ pivot.
i $\leftarrow$ i + 1
END while.

Step 8:- If i < j then
swap A[i], A[j]

Step 9:- END If
END while.

Step 10:- swap A [low], A[j]

Step 11:- Return .J

Step 12:- END.

Program:

```
#include <iostream>
using namespace std;
```

```
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
```

```
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[] = {34, 7, 23, 32, 5, 62};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
```

```
cout << endl;
```

```
return 0;
```

```
}
```

Output:

```
Original array: 34 7 23 32 5 62
Sorted array:   5 7 23 32 34 62
```

Time complexity:

Time complexity:-

1) Best case:- Recurrence $T(n) = 2T(n/2) + O(n)$

$$T(n) = n \log_2 n$$

$$T(n) = n(n \log_2 n)$$

2) Avg case:- $T(n) = 2T(n/2) + n$

$$= 2[2T(n/4) + n/2] + n$$

$$= 4T(n/4) + 2n$$

$$= 8T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n$$

$$k = \log_2 n$$

$$T(n) = n \log_2 n$$

$$T(n) = O(n \log n)$$

3) Worst case:-

$$T(n) = n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

$$= \frac{n^2 + n}{2}$$

$$T(n) = O(n^2)$$

Conclusion:

Quick sort sorts the array by selecting a pivot element and partitioning the array into two parts. It recursively sorts the subarrays and arranges the elements in ascending order efficiently.